# Monitoring and Operations Guide

CloudWatch, ECS, ALB, DocumentDB and runbooks

Flipkart Clone Microservices Workspace

# Monitoring and Operations Guide

This guide explains how to monitor the Flipkart Clone locally and on AWS after deployment.

It covers:

- Local Docker monitoring
- ECS service monitoring
- CloudWatch logs and metrics
- ALB and target group health
- DocumentDB monitoring
- CloudWatch dashboards
- CloudWatch alarms
- Logs Insights queries
- Incident troubleshooting runbooks
- Daily and weekly operations checklist

# PART 1: Monitoring Locally with Docker

## Check Container Status

```
cd /home/user/flipkart-clone
docker compose ps
```

All services should show `running` or `healthy`.

## View All Logs

```
docker compose logs -f
```

## View One Service Log

```
docker compose logs -f api-gateway
```

Other examples:

```
docker compose logs -f product-service
docker compose logs -f user-service
docker compose logs -f mongodb
```

## Check Health Endpoints

| Service | Local Health URL |
|---|---|
| API Gateway | `http://localhost:3000/health` |
| User Service | `http://localhost:3001/health` |
| Product Service | `http://localhost:3002/health` |
| Cart Service | `http://localhost:3003/health` |
| Order Service | `http://localhost:3004/health` |
| Payment Service | `http://localhost:3005/health` |
| Notification Service | `http://localhost:3006/health` |
| Frontend | `http://localhost:8080/healthz` |

Test quickly:

```
curl http://localhost:3000/health
curl http://localhost:3000/api/products?limit=3
```

## Local Smoke Test

```
./scripts/smoke-test.sh
```

# PART 2: AWS Monitoring Overview

Production monitoring should include these layers:

| Layer | AWS Service |
|---|---|
| Container logs | CloudWatch Logs |
| CPU and memory | ECS CloudWatch Metrics |
| Request/latency/5XX | Application Load Balancer Metrics |
| Database | DocumentDB Metrics |
| Service health | ECS Service Events and Target Groups |
| Dashboards | CloudWatch Dashboards |
| Alerts | CloudWatch Alarms + SNS |
| Tracing | AWS X-Ray optional |
| Security traffic | WAF logs optional |
| Cost | AWS Budgets and Cost Explorer |

# PART 3: ECS Service Monitoring

## Where to Check ECS Health

1. Open AWS Console.
2. Go to **ECS**.
3. Open cluster: `flipkart-cluster`.
4. Open **Services** tab.
5. Check each service:

- `frontend-service`
- `api-gateway-service`
- `user-service`
- `product-service`
- `cart-service`
- `order-service`
- `payment-service`
- `notification-service`

## What Healthy Looks Like

For each service:

| Field | Healthy Value |
|---|---|
| Desired tasks | 1 or 2 depending setup |
| Running tasks | Same as desired |
| Pending tasks | 0 |
| Failed deployments | 0 |
| Latest deployment | Completed or steady state |

## ECS Service Events

Open each ECS service and check **Events**.

Important error messages:

| Error | Meaning |
|---|---|
| CannotPullContainerError | ECR image, tag, or IAM issue |
| Essential container exited | App crashed after startup |
| Health checks failed | ALB or app health endpoint issue |
| ResourceInitializationError | Secrets/logs/network issue |
| Cannot resolve host | Cloud Map or DNS issue |

# PART 4: CloudWatch Logs

## Recommended Log Groups

Create or confirm these log groups:

```
/ecs/flipkart-api-gateway
/ecs/flipkart-user-service
/ecs/flipkart-product-service
/ecs/flipkart-cart-service
/ecs/flipkart-order-service
/ecs/flipkart-payment-service
/ecs/flipkart-notification-service
/ecs/flipkart-frontend
```

## Set Log Retention

For development:

```
7 days
```

For production:

```
30 to 90 days
```

Steps:

1. Open **CloudWatch**.
2. Go to **Logs** → **Log groups**.
3. Select log group.
4. Click **Actions** → **Edit retention setting**.
5. Choose retention period.

## Logs Insights Queries

### Find Errors

```
fields @timestamp, @message
| filter @message like /ERROR|Error|error|failed|Failed/
| sort @timestamp desc
| limit 100
```

### API Gateway 502 or Proxy Problems

```
fields @timestamp, @message
| filter @message like /Proxy error|Bad gateway|ECONNREFUSED|ETIMEDOUT/
| sort @timestamp desc
| limit 100
```

### MongoDB Connection Problems

```
fields @timestamp, @message
| filter @message like /MongoDB|Mongoose|connection|ECONNREFUSED|authentication failed/
| sort @timestamp desc
| limit 100
```

## Authentication Problems

```
fields @timestamp, @message
| filter @message like /Invalid token|Authentication|JWT|401/
| sort @timestamp desc
| limit 100
```

## Slow Requests if Logs Include Timing

```
fields @timestamp, @message
| filter @message like /response time|duration|slow/
| sort @timestamp desc
| limit 50
```

# PART 5: Application Load Balancer Monitoring

## Where to Check

1. Go to **EC2**.
2. Click **Load Balancers**.
3. Select `flipkart-alb`.
4. Check tabs:

• Monitoring

• Listeners

• Security

• Target groups

## Important ALB Metrics

| Metric | Watch For |
|---|---|
| `RequestCount` | Traffic volume |
| `TargetResponseTime` | API/frontend latency |
| `HTTPCode_ELB_5XX_Count` | Load balancer errors |
| `HTTPCode_Target_5XX_Count` | App/service errors |
| `HTTPCode_Target_4XX_Count` | Bad requests/auth errors |
| `HealthyHostCount` | Healthy ECS tasks |
| `UnHealthyHostCount` | Unhealthy ECS tasks |

## Recommended ALB Alarm Thresholds

| Alarm | Threshold |
|---|---|
| ALB 5XX | > 10 in 5 minutes |
| Target 5XX | > 20 in 5 minutes |
| Response time | > 2 seconds for 5 minutes |
| Unhealthy targets | > 0 for 2 periods |

## Target Group Health

Open **EC2** → **Target Groups**:

- `flipkart-api-tg`
- `flipkart-frontend-tg`

Targets should be **healthy**.

If unhealthy, check:

• Container port mapping

• Health check path

- Security group from ALB to service
- ECS task logs
- App is listening on correct port

# PART 6: DocumentDB Monitoring

Open **DocumentDB** → `flipkart-docdb-cluster` → **Monitoring**.

## Key Metrics

| Metric | Meaning | Alert When |
|---|---|---|
| CPUUtilization | Database CPU | > 75-80% |
| DatabaseConnections | Active connections | Sudden spikes or near max |
| FreeableMemory | Available memory | Very low for sustained period |
| ReadThroughput | Read traffic | Unexpected spikes |
| WriteThroughput | Write traffic | Unexpected spikes |
| ReadLatency | Read delay | Increasing over baseline |
| WriteLatency | Write delay | Increasing over baseline |
| DiskQueueDepth | Disk pressure | Sustained high value |

## Common DocumentDB Problems

### Too Many Connections

Fix:

- Use connection pooling.
- Ensure each service creates one Mongoose connection, not one per request.
- Scale database instance if needed.

### Slow Product Search

Fix:

- Add indexes.
- Cache frequent reads in Redis.
- Avoid unbounded queries.

### Authentication Failed

Fix:

- Verify username/password.
- Check Secrets Manager value.
- Recreate ECS task with updated secret.

# PART 7: CloudWatch Dashboard Setup

Create dashboard:

1. Open **CloudWatch**.
2. Go to **Dashboards**.
3. Click **Create dashboard**.
4. Name: `Flipkart-Monitoring`.

## Recommended Widgets

### ECS CPU Widget

Metrics:

```
ECS → ClusterName, ServiceName → CPUUtilization
```

Include all services.

### ECS Memory Widget

Metrics:

```
ECS → ClusterName, ServiceName → MemoryUtilization
```

### ALB Traffic Widget

Metrics:

```
ApplicationELB → RequestCount
ApplicationELB → TargetResponseTime
ApplicationELB → HTTPCode_Target_5XX_Count
ApplicationELB → HTTPCode_ELB_5XX_Count
```

### Target Health Widget

Metrics:

```
HealthyHostCount
UnHealthyHostCount
```

### DocumentDB Widget

Metrics:

```
CPUUtilization
DatabaseConnections
FreeableMemory
ReadLatency
WriteLatency
```

# PART 8: CloudWatch Alarms

Create SNS topic first:

```
flipkart-alerts
```

Subscribe your email and confirm the email subscription.

## Recommended Alarms

| Alarm Name | Metric | Condition |
|---|---|---|
| `API-Gateway-High-CPU` | ECS CPU | > 80% for 5 min |
| `Product-Service-High-Memory` | ECS Memory | > 85% for 5 min |
| `ALB-Target-5XX-High` | Target 5XX | > 20 in 5 min |
| `ALB-Unhealthy-Targets` | UnHealthyHostCount | > 0 for 2 periods |
| `DocumentDB-High-CPU` | DocDB CPU | > 80% for 10 min |
| `DocumentDB-Low-Memory` | FreeableMemory | Below safe baseline |
| `CloudWatch-Log-Errors` | Metric filter | Error count > threshold |

## Create Metric Filter for Errors

1. Open a CloudWatch log group.
2. Click **Metric filters**.
3. Create filter pattern:

```
?ERROR ?Error ?error ?failed ?Failed
```

4. Create metric namespace:

```
FlipkartApp
```

5. Metric name:

```
ServiceErrorCount
```

6. Create alarm on this metric.

# PART 9: Deployment Monitoring

When deploying a new version:

1. Open ECS service.
2. Click **Deployments and events**.
3. Watch new tasks start.
4. Confirm old tasks drain.
5. Confirm target groups become healthy.
6. Watch CloudWatch logs.
7. Test health endpoints.
8. Test frontend in browser.

## Deployment Verification Commands

From your local machine:

```
curl http://ALB_DNS_NAME/api/health
curl http://ALB_DNS_NAME/api/products?limit=3
```

If HTTPS/domain is configured:

```
curl https://yourdomain.com/api/health
curl https://yourdomain.com/api/products?limit=3
```

# PART 10: Incident Runbooks

## Runbook A: Frontend Not Opening

Check in order:

1. ALB DNS opens or not.
2. Frontend target group healthy.
3. Frontend ECS tasks running.
4. Frontend service logs.
5. Security group allows ALB to frontend port 80.
6. Nginx health endpoint `/healthz`.
7. Browser console errors.

## Runbook B: API Returns 502

Check:

1. API target group health.
2. API Gateway ECS tasks running.
3. API Gateway logs.
4. API Gateway env vars point to correct Cloud Map names.
5. Backend services are running.
6. Security groups allow API Gateway to services.
7. Cloud Map DNS namespace is correct.

## Runbook C: Login/Register Fails

Check:

1. User service logs.
2. DocumentDB connectivity.
3. JWT secret present.
4. MongoDB URI has correct database name.
5. API request body is valid JSON.
6. User collection has unique email index.

## Runbook D: Products Not Showing

Check:

1. Product service running.
2. Product service logs.
3. Product database has seeded products.

4. Product route is `/api/products`.
5. Frontend API base URL is correct.

If database is empty, seed products locally before image build or create an admin seed task.

## Runbook E: Cart Fails

Check:

1. Cart service logs.
2. Product service reachable from cart service.
3. `PRODUCT_SERVICE_URL` env var.
4. JWT token is present in request.
5. User ID matches logged-in user.

## Runbook F: Order Checkout Fails

Check:

1. Order service logs.
2. Cart has items.
3. Cart service reachable from order service.
4. Notification service reachable.
5. Payment service logs.
6. DocumentDB connection for orders and payments.

# PART 11: Cost Monitoring

Use:

- AWS Budgets
- Cost Explorer
- CloudWatch log retention
- Tags

Recommended tags:

```
Project=FlipkartClone
Environment=Dev or Prod
Owner=YourName
Service=api-gateway or user-service etc.
```

High-cost areas to watch:

| Service | Why Cost Rises |
|---|---|
| NAT Gateway | Hourly cost and data transfer |
| DocumentDB | Instance hours and backups |
| CloudWatch Logs | High log ingestion and long retention |
| Fargate | Too many tasks or large CPU/memory |
| WAF | Paid managed rules or high request volume |
| DataDog | Per-host/APM pricing |

# PART 12: Daily Operations Checklist

Do this daily in production:

- [ ] Check ECS services all running.
- [ ] Check ALB target groups healthy.
- [ ] Check CloudWatch alarms state.
- [ ] Review last 24 hours error logs.
- [ ] Check DocumentDB CPU and connections.
- [ ] Check deployment events if any release happened.
- [ ] Check AWS Budget status.

# PART 13: Weekly Operations Checklist

Do this weekly:

- [ ] Review CloudWatch dashboard trends.
- [ ] Review slow endpoints and high latency.
- [ ] Confirm backups are working.
- [ ] Check ECR old image cleanup policy.
- [ ] Check IAM users and keys.
- [ ] Rotate secrets if needed.
- [ ] Review WAF blocked requests.
- [ ] Review Cost Explorer by service.
- [ ] Confirm log retention settings.

# PART 14: Production Monitoring Minimum Setup

If you want a minimum production setup, configure these at least:

1. ECS Container Insights.
2. CloudWatch log groups for every service.
3. ALB 5XX alarm.
4. ALB unhealthy target alarm.
5. ECS CPU and memory alarms.
6. DocumentDB CPU and connection alarms.
7. SNS email alert topic.
8. AWS Budget alert.
9. CloudWatch dashboard.

This gives enough visibility to operate the application safely.